GraphQL Quick Start Guide

Einführung

GraphQL ist eine Abfragesprache für APIs und eine Laufzeitumgebung zur Ausführung dieser Abfragen. Im Gegensatz zu REST ermöglicht GraphQL präzise Datenabfragen, bei denen Clients genau die Daten anfordern können, die sie benötigen - nicht mehr und nicht weniger. Diese Anleitung hilft Ihnen, schnell mit der Hero Software GraphQL-API zu starten und effektive Schnittstellen zu entwickeln.

Inhaltsverzeichnis

- Einführung
- Inhaltsverzeichnis
- Zugriff auf die API-Dokumentation
  - Hero Software GraphQL-Endpoint
  - Introspection nutzen
  - Zugriff mit Authentication Token
  - Schema-Explorer in GraphQL-Clients
  - Vorteile der integrierten Dokumentation
  - Rate Limiting und Best Practices
- GraphQL-Grundlagen
- Einrichtung der Entwicklungsumgebung
  - Altair GraphQL Client
  - Insomnia
- Abfragen (Queries)
  - Grundlegende Query-Syntax
  - Beispiel: Kunden suchen
  - Beispiel: Projekte suchen
  - Abfragen mit Filtern
- Mutationen
  - Grundlegende Mutations-Syntax
  - Beispiel: Kontakt erstellen
  - Beispiel: Kontakt aktualisieren
  - Beispiel: Aufgabe erstellen
  - Beispiel: Projekt erstellen
- Schema und Typen
  - Haupttypen in der Hero Software API
  - Nützliche Abfragen für Metadaten
- Best Practices
  - Effiziente Abfragen
  - Namenskonventionen
  - Fehlerbehandlung
- Fehlerbehebung
  - Häufige Probleme und Lösungen
  - Debugging-Tipps

- Weiterführende Ressourcen
- Fazit

Zugriff auf die API-Dokumentation

Ein großer Vorteil von GraphQL ist die integrierte Selbstdokumentation. Das vollständige API-Schema mit allen verfügbaren Typen, Feldern und Operationen ist direkt über den GraphQL-Endpunkt zugänglich.

Hero Software GraphQL-Endpoint

Der offizielle GraphQL-Endpoint der Hero Software ist unter folgender URL erreichbar:

```
https://login.hero-software.de/api/external/v9/graphql
```

Dies ist der Produktions-Endpoint für alle externen API-Zugriffe. Stellen Sie sicher, dass Sie diesen Endpoint für alle Ihre Integrationen verwenden.

Introspection nutzen

GraphQL bietet eine sogenannte "Introspection"-Funktion, mit der Sie das Schema abfragen können:

```
query {
  __schema {
    types {
      name
      description
      fields {
        name
        description
      }
    }
  }
}

```

Zugriff mit Authentication Token

Um auf den Hero Software GraphQL-Endpunkt zuzugreifen, benötigen Sie einen gültigen Authentication Token:

1. Melden Sie sich in Ihrem Hero Software-Konto an
2. Navigieren Sie zu "Einstellungen" > "API-Zugriff"
3. Generieren Sie einen API-Token (falls noch nicht vorhanden)
4. Verwenden Sie diesen Token in Ihren API-Anfragen im Authorization-Header:

```
Authorization: Bearer IHR_TOKEN_HIER
```

Beispiel für eine Anfrage mit cURL:

```
curl -X POST \
  https://login.hero-software.de/api/external/v9/graphql \
  -H 'Authorization: Bearer IHR_TOKEN_HIER' \
  -H 'Content-Type: application/json' \
  -d '{"query":"{ user { id email } }"}'
```

Schema-Explorer in GraphQL-Clients

Sowohl Altair als auch Insomnia bieten integrierte Schema-Explorer:

- **In Altair**: Klicken Sie auf den "Docs"-Tab in der rechten Seitenleiste, um das vollständige Schema zu durchsuchen. Geben Sie die Hero Software GraphQL-URL ein und fügen Sie den Authorization-Header hinzu.
- **In Insomnia**: Klicken Sie auf "Schema" im GraphQL-Anfragefenster, um die Dokumentation anzuzeigen. Konfigurieren Sie die Anfrage mit der Hero Software GraphQL-URL und dem entsprechenden Authorization-Header.

Diese Tools laden das Schema automatisch, wenn Sie eine Verbindung zum GraphQL-Endpunkt herstellen und ermöglichen es Ihnen, alle verfügbaren Typen, Felder, Queries und Mutations zu erkunden.

Vorteile der integrierten Dokumentation

- **Immer aktuell**: Die Dokumentation wird direkt aus dem Schema generiert und ist daher immer auf dem neuesten Stand
- **Vollständig**: Alle Typen, Felder und Operationen sind dokumentiert
- **Kontextuell**: Die Dokumentation ist direkt während der Entwicklung verfügbar
- **Autovervollständigung**: GraphQL-Clients können Vorschläge basierend auf dem Schema anbieten

Rate Limiting und Best Practices

Die Hero Software API unterliegt einem Rate Limiting. Beachten Sie folgende Best Practices:

- Verwenden Sie Caching für häufig abgefragte Daten
- Bündeln Sie mehrere Abfragen in einer GraphQL-Anfrage
- Vermeiden Sie unnötige Polling-Anfragen
- Implementieren Sie eine Exponential-Backoff-Strategie bei Fehlern

GraphQL-Grundlagen

GraphQL basiert auf drei Hauptkonzepten:

1. **Queries**: Daten abrufen
2. **Mutations**: Daten erstellen, aktualisieren oder löschen

3. **Schema**: Definition der verfügbaren Datentypen und Operationen

Der größte Vorteil von GraphQL ist die Möglichkeit, genau die benötigten Daten in einer einzigen Anfrage zu erhalten, was die Netzwerkeffizienz verbessert und Überabrufe vermeidet.

Einrichtung der Entwicklungsumgebung

Für die Arbeit mit der Hero Software GraphQL-API empfehlen wir folgende Tools:

Altair GraphQL Client

Altair ist ein leistungsstarker, kostenloser GraphQL-Client mit einer benutzerfreundlichen Oberfläche.

Vorteile:

- Speziell für GraphQL optimiert
- Unterstützt Schema-Introspection und Autovervollständigung
- Verfügbar als Desktop-App und Browser-Erweiterung
- Unterstützt mehrere Abfragen in verschiedenen Fenstern
- Einfache Header-Verwaltung

Installation:

1. Besuchen Sie altair.sirmuel.design
2. Laden Sie die Version für Ihr Betriebssystem herunter oder installieren Sie die Browser-Erweiterung

Insomnia

Insomnia ist ein vielseitiger API-Client, der sowohl REST als auch GraphQL unterstützt.

Vorteile:

- Unterstützt verschiedene API-Typen (REST, GraphQL, gRPC)
- Umgebungsvariablen und Request-Verkettung
- Gute Teamkollaborationsfunktionen
- Integrierte Authentifizierungsunterstützung

Installation:

1. Besuchen Sie insomnia.rest
2. Laden Sie die kostenlose Version herunter und installieren Sie sie

Abfragen (Queries)

GraphQL-Queries ermöglichen es Ihnen, genau die Daten abzurufen, die Sie benötigen. Die Struktur einer Query spiegelt die Struktur der zurückgegebenen Daten wider.

Grundlegende Query-Syntax

```
query {
  feldname {
    unterfeldA
    unterfeldB
  }
}
```

Beispiel: Kunden suchen

```
query MyQuery {
  contacts(search: "Max Mustermann") {
    id
    first_name
    last_name
    email
    phone_home
    phone_mobile
    address {
      street
      city
      zipcode
      country {
        id
        name
      }
    }
    project_matches {
      id
      name
      volume
      project_nr
      current_project_match_status {
        name
      }
    }
  }
}

```

Diese Query sucht nach Kontakten mit dem Namen "Michael Klaus" und gibt ausgewählte Informationen zurück.

Beispiel: Projekte suchen

```
query FindProjectsWithSearch {
  project_matches(search: "Schiffe") {
    id
    name
    project {
```

```
      id
      name
    }
    customer {
      id
      company_name
      first_name
      last_name
      address {
        street
        city
        zipcode
        country {
          name
        }
      }
    }
    address {
      street
      city
      zipcode
      country {
        name
      }
    }
    contact {
      id
      first_name
      last_name
    }
  }
}

```

Abfragen mit Filtern

Die Hero Software API bietet verschiedene Filter für Abfragen:

```
query MyQuery {
  tasks(
    project_match_id: 10
    target_user_ids: 10
    is_done: false
    show_deleted: false
  ) {
    id
    due_date
    done_date
    title
    comment
  }
}

```

Wichtig: Bei der Verwendung von orderBy können nur einfache Datenfelder verwendet werden, nicht mit "DESC" oder "ASC". Beispiel: orderBy: "created" ist korrekt, orderBy: "DESC created"

ist falsch.

Mutationen

Mutationen werden verwendet, um Daten zu erstellen, zu aktualisieren oder zu löschen.

Grundlegende Mutations-Syntax

```
mutation {
  mutationsName(parameter: wert) {
    rückgabefeld
  }
}

```

Beispiel: Kontakt erstellen

```
mutation MyMutation {
  create_contact(
    contact: {
      first_name: "Test"
      last_name: "Test"
      title: "Herr"
      address: {
        city: "Lindhorst"
        street: "Danziger Str. 7"
        zipcode: "31690"
        country_id: 1
      }
      email: "michael.kraus@hero-software.de"
      type: "private"
      phone_mobile: "+491761234567"
      phone_home: "+491761234567"
    }
  ) {
    id
  }
}

```

Beispiel: Kontakt aktualisieren

```
mutation MyMutation {
  update_contact(
    contact: {
      id: 4667391
      first_name: "Test"
      last_name: "Test"
      title: "Herr"
      address: {
        city: "Lindhorst"
        street: "Danziger Str. 7"
        zipcode: "31690"
        country_id: 1
      }
      email: "michael.klaus@hero-software.de"
      type: "private"
```

```
      phone_mobile: "+491761234567"
      phone_home: "+491761234567"
    }
  ) {
    id
  }
}

```

Beispiel: Aufgabe erstellen

```
mutation MyMutation {
  update_task(
    task: {
      title: "Titel"
      comment: "Kommentar"
      due_date: "2025-05-26T12:31:47+00:00"
      target_project_match_id: 6547857
      target_user_id: 259159
    }
  ) {
    id
  }
}

```

Beispiel: Projekt erstellen

```
mutation CreateProjectMatch {
  create_project_match(
    project_match: {
      name: "Name des Projekts"
      customer_id: 3760080
      contact_id: 3760080
      type_id: 15461
      step_id: 195574
      address_id: 8692643
    }
  ) {
    id
    name
    customer {
      id
      company_name
    }
    type {
      id
      name
    }
  }
}

```

Schema und Typen

Das GraphQL-Schema definiert die Struktur Ihrer API, einschließlich der verfügbaren Typen, Felder und Operationen.

Haupttypen in der Hero Software API

- **Query**: Einstiegspunkt für Datenabfragen
- **Mutation**: Einstiegspunkt für Datenänderungen
- **Customer**: Repräsentiert Kundendaten
- **ProjectMatch**: Repräsentiert Projektdaten
- **Task**: Repräsentiert Aufgaben
- **CalendarEvent**: Repräsentiert Termine

Nützliche Abfragen für Metadaten

Um verfügbare Typen und Optionen zu erkunden:

```
query ListProjectTypes {
  project_types {
    id
    name
    project_status_steps {
      id
      is_active
      name
    }
  }
}

```

```
query MyQuery {
  calendar_event_categories {
    id
    name
  }
}

```

Best Practices

Effiziente Abfragen

1. **Nur benötigte Felder anfordern**: Vermeiden Sie das Abrufen unnötiger Daten.
2. **Verschachtelte Abfragen optimieren**: Achten Sie auf die Tiefe Ihrer Abfragen.
3. **Pagination verwenden**: Nutzen Sie first, last und offset für große Datensätze.

```
query {
  contacts(first: 10, offset: 0) {
    id
    first_name
```

```
    last_name
  }
}

```

Namenskonventionen

1. **Konsistente Benennung**: Verwenden Sie beschreibende Namen für Ihre Operationen.
2. **Operationstyp angeben**: Kennzeichnen Sie Ihre Operationen mit query oder mutation.

```
query FindContactByName {
  contacts(search: "Name") {
    id
    first_name
  }
}

```

Fehlerbehandlung

GraphQL gibt Fehler im errors-Feld der Antwort zurück. Implementieren Sie eine robuste Fehlerbehandlung in Ihrem Client.

Fehlerbehebung

Häufige Probleme und Lösungen

1. **Syntaxfehler**: Überprüfen Sie die Klammern und Anführungszeichen in Ihren Abfragen.
2. **Feldnamen nicht gefunden**: Stellen Sie sicher, dass die angeforderten Felder im Schema existieren.
3. **Authentifizierungsprobleme**: Überprüfen Sie Ihre Authentifizierungsheader.

Debugging-Tipps

1. **Schema-Introspection**: Nutzen Sie die Introspektionsfunktionen von Altair oder Insomnia.
2. **Schrittweise Entwicklung**: Bauen Sie komplexe Abfragen schrittweise auf.
3. **Variablen verwenden**: Nutzen Sie Variablen für wiederholte Werte.

Weiterführende Ressourcen

- Offizielle GraphQL-Dokumentati...
- Apollo GraphQL Tutorials
- Altair GraphQL Client Dokument...
- Insomnia GraphQL Support

Fazit

GraphQL bietet eine flexible und effiziente Möglichkeit, mit APIs zu interagieren. Mit diesem Quick Start Guide können Sie schnell mit der Hero Software GraphQL-API arbeiten und leistungsstarke Integrationen entwickeln. Durch die Nutzung der richtigen Tools wie Altair oder Insomnia und die Befolgung der Best Practices werden Sie in der Lage sein, effiziente und wartbare GraphQL-Implementierungen zu erstellen.

Beginnen Sie mit einfachen Abfragen und erweitern Sie Ihre Kenntnisse schrittweise, um das volle Potenzial von GraphQL auszuschöpfen.